Title Page

Block MVP for Nova Services — Software BOM + Dependency Plan

Dependency-accurate build plan from ZERO → usable MVP

Target date: March 1, 2026

Work start: Monday, Feb 16, 2026

Team capacity assumption: 2 devs × 6 productive hrs/day × 10 weekdays = 120 hrs

Document Date:	Feb 8, 2026
Audience:	Internal engineering plan (small team)
Tenant Scope (MVP):	Nova Services only (single-tenant acceptable)
Platforms:	Next.js (Web), React Native/Expo (Mobile)
Data + Auth:	Supabase (Postgres, Auth, Storage)
Maps:	Mapbox (Web + RN)
Payments:	Stripe
Calendar:	Google Calendar
Email:	Postmark

Contents

Title Page **1**

Executive Summary **2**

System Overview **2**

Data Model (Conceptual + Implementation Notes) **3**

Task BOM **4**

Dependency Graphs (DAG) **11**

Milestones & Timeline **22**

Effort Summary **23**

Machine-readable Dependency Block **24**

Appendix **25**

Executive Summary

This document is a dependency-accurate Software Bill of Materials (BOM) for building **Block MVP for Nova Services** from **zero** to a usable MVP by **March 1, 2026**. It includes:

- A complete task BOM (Epics → Stories → Tasks) with stable IDs, owners, estimates, and deliverables.
- Dependency graphs (DAGs) in TikZ and a complete dependency table for cross-checking.
- A milestone plan aligned to the available capacity window (Feb 16–Feb 27) and a contingency cutline.
- A machine-readable JSON dependency block (valid JSON) for importing into tooling.

MVP end-to-end usability (must work): manual lead creation → map view → polygon boundaries → clustering → assign clusters to reps → reps log knock outcomes → quote (price-per-sqft + adjustments) → accept quote into job → schedule to crew → crew views schedule, completes checklist, captures before/after photos (offline upload queue) → generate invoice → take payment via Stripe → record ledger entry → send email notifications → audit log key actions → Google Calendar appointment sync.

Capacity reality: The P0 plan below is engineered to **exactly match** the stated capacity of **120 hours**. This implies **near-zero schedule slack**. The cutline section identifies what to drop or reduce first to protect the release.

System Overview

Scope Summary (MVP)

- **Tenant:** Nova Services only (single tenant acceptable for MVP).
- **Roles (RBAC):** Admin, Manager, Sales Rep, Labor/Crew.
- **Workflows (must work end-to-end):**
  - **A)** Create leads (manual) → map → polygon boundaries → cluster leads by business rules.
  - **B)** Assign clusters to reps → track knocks/outcomes.
  - **D)** Schedule jobs to crews → track status → completion photos.
  - **E)** Invoice/payment tracking (in-house accounting basics; no QuickBooks).
- **Integrations (MVP):** Email (yes), Stripe (yes), Google Calendar (yes), SMS/Calling (no).

Architecture

Backend approach (chosen, single approach): Next.js API Routes (TypeScript) hosted alongside the web app.

- **Justification:** Minimizes services to ship, reduces operational overhead, and provides a secure place for secret-backed operations (Stripe webhooks, Postmark sends, Google OAuth/token refresh, clustering operations requiring server validation). CRUD reads/writes primarily use Supabase with RLS; API routes are used for privileged and integration flows.

Stack

- **Web:** Next.js (TypeScript), online-only.
- **Mobile:** React Native (TypeScript) via Expo; graceful degradation (cache last sync, offline photo queue).
- **Auth + DB:** Supabase (Postgres + Auth + RLS).
- **File Storage:** Supabase Storage (object storage) for photos/files; signed uploads where needed.
- **Maps:** Mapbox GL JS (web) + Mapbox RN SDK (mobile).
- **Payments:** Stripe Checkout + webhooks.
- **Calendar:** Google Calendar API (OAuth connect; one-way sync for appointment records).
- **Email:** Postmark (transactional).
- **Hosting:** Vercel for web + API routes; Expo/EAS for internal distribution of mobile builds.

Non-Functional Requirements (MVP)

- **Security:** RBAC aligned to roles; least privilege via Supabase RLS; server secrets only in API routes.
- **Auditability:** Record key actions (lead creation, assignment, quote changes, job status changes, invoice/payment events).
- **Performance:** Small-business scale (thousands of leads, hundreds of jobs) using indexed relational model.
- **Reliability:** Mobile caches last synced data; offline photo upload queue with retry.
- **Data Integrity:** Strong FK constraints, status enums, clear ownership + timestamps.

Data Model (Conceptual + Implementation Notes)

Conceptual Entities (MVP)

- **Property/Address** (lat/lng, sqft, normalized address fields)
- **Lead** (status, source=manual, linked to Property)
- **Customer** (contact info)
- **Cluster** (polygon boundary stored as GeoJSON; business-rule metadata; assignment to rep user)
- **Knock** (lead outcome events: knocked, no answer, not interested, interested, booked estimate, sold)
- **Quote** + **Quote Items** (price-per-sqft base + adjustment line items)
- **Job** (created from accepted Quote; status; assigned crew)
- **Appointment** (schedule record; optional Google Calendar event id)
- **Crew** (+ crew members mapped to user accounts)
- **Invoice** (snapshot from job/quote; status lifecycle)
- **Payment** (Stripe-backed; reconciliation fields)
- **Attachments** (Notes/Photos/Files; stored in object storage; references parent record)
- **Ledger Entries** (in-house accounting basics: invoice issued, payment received)
- **Audit Log** (key actions)

Implementation Notes

- **No standalone "Territory" entity:** polygons/boundaries are stored on **Cluster** (e.g., `clusters.boundary_ge`
- **Geo fields:** store `properties.lat`, `properties.lng` (numeric) and `clusters.boundary_geojson` (jsonb). Clustering can be computed in the API route using business rules (max radius, max items per cluster).
- **Status enums (examples):**
  - Lead: `new, contacted, qualified, won, lost`
  - Quote: `draft, sent, accepted, declined`
  - Job: `new, scheduled, in_progress, complete, invoiced, paid`
  - Invoice: `draft, sent, paid, void`
- **RLS (role-aligned):**
  - Admin/Manager: full read/write across tenant.
  - Sales Rep: read clusters assigned to them; read leads in assigned clusters; write knocks for those leads; create/edit quotes for those leads.
  - Labor/Crew: read jobs/appointments assigned to their crew; write checklist completion and upload attachments for those jobs.
- **Audit logging:** application-level inserts into `audit_log` for key transitions, plus optional DB function for consistent formatting.
- **Ledger basics:** use a simple `ledger_entries` table with references to invoices/payments and double-entry style fields (debit/credit accounts) sufficient for MVP bookkeeping.

Task BOM

Epics

Epic ID	Epic Name	MVP P ity
E01	Foundations & Delivery (repo, env, CI/CD, deploy)	P0
E02	Auth, RBAC, RLS (roles, policies, guarded UX)	P0
E03	Data Model & Cross-cutting (schema, storage, audit, ledger)	P0
E04	Maps & Clustering (Mapbox, polygons, clustering rules)	P0
E05	Sales Workflow (leads, assignment, knocks, quotes, job creation)	P0
E06	Ops Workflow (crews, scheduling, crew app, checklist, photos)	P0
E07	Billing & Payments (invoice, Stripe, email notifications)	P0
E08	Integrations & Release (Calendar, dashboards, observability, release)	P0
E09	MVP2 Backlog (enhancements, scaling, advanced features)	P1/P2

Stories (Sequential IDs)

Story ID	Epic	Story Name

S01	E01	Monorepo initialization and core tooling
S02	E01	Environments, secrets, CI/CD, deploy
S03	E03	Core schema migrations (entities, enums, indexes)
S04	E03	Storage and attachments (buckets, policies, upload patterns)
S05	E03	Audit logging and in-house ledger primitives
S06	E02	RLS policies and permission helpers
S07	E02	User bootstrap and authentication UX (web + mobile)
S08	E04	Web Mapbox map foundation
S09	E04	Polygon drawing/editing and persistence
S10	E04	Clustering engine and cluster management
S11	E05	Lead management (manual entry, list/search)
S12	E05	Cluster assignment and knock tracking
S13	E05	Quote-to-job flow (price-per-sqft + adjustments)
S14	E06	Scheduling, crews, and crew app foundation
S15	E06	Photos/attachments capture and viewing
S16	E07	Billing, payments, and email notifications
S17	E08	Google Calendar appointment sync
S18	E08	Operations dashboard (Nova)
S19	E08	Observability and release readiness
S20	E09	Mobile sales rep app (optional)
S21	E09	Imports and data enrichment
S22	E09	Reporting and customer portal enhancements
S23	E09	Observability and QA expansion
S24	E09	Multi-tenant and advanced optimizations

Task BOM Table (All Tasks)

Task ID	Task Name	Epic	Story	Subsystem	Owner	Description / Definition of Done	Inputs/Outputs	Dependencies	Hours	Pri	Deliverables
T001	Initialize monorepo	E01	S01	DevOps	DevOps	Create repo structure for `apps/web` (Next.js), `apps/mobile` (Expo RN), `packages/shared`; builds run locally; initial README.	I/O: mono layout; shared package exports.		4	P0	Repo skeleton R README.
T002	Configure code quality toolchain	E01	S01	DevOps	DevOps	TypeScript strict mode, ESLint, Prettier, consistent scripts; lint passes; formatting enforced; basic conventions documented.	I/O: `tsconfig`, lint rules, formatter config; `npm` scripts.	`T001`	3	P0	Lint/format c tor; notes.
T003	Set up env/secrets patterns	E01	S02	DevOps	DevOps	Define local/dev/prod env variable schema; `.env.example` files; secrets handling documented; app boots with placeholders.	I/O: env keys for Supabase/Mapbox/Stripe/Postmark/Google.	`T001`	2	P0	`.env.example` helper; docs.
T004	Provision Supabase + local dev	E01	S02	DevOps	DevOps	Create Supabase projects (dev/prod), enable local Supabase for development; connectivity verified; storage enabled.	I/O: Supabase URLs/keys; local CLI workflow.	`T003`	3	P0	Supabase proj structions; sto
T005	Set up CI + preview deploy	E01	S02	DevOps	DevOps	GitHub Actions runs lint/typecheck/tests/build; preview deploy configured; failures block merge.	I/O: CI pipeline; preview env.	`T002, T003`	3	P0	CI workflows; deploy notes.
T006	Configure production deploy	E01	S02	DevOps	DevOps	Vercel production env wired; API routes run; release config stored; mobile build profiles documented (internal distribution).	I/O: Vercel projects; Expo build profiles.	`T005`	3	P0	Vercel prod c list; draft; mob
T007	Implement core DB schema migrations	E03	S03	DB	BE	Create migrations for MVP entities + enums + FK constraints + indexes; schema applies cleanly in dev; seed minimal reference data.	I/O: Postgres tables/enums; migrations; seed script.	`T004`	5	P0	SQL migrati notes; seed scr
T008	Implement storage + attachments upload flow	E03	S04	Integrations	BE	Create storage buckets and access policies; implement signed upload pattern (where needed); mobile/web can upload and store attachment metadata.	I/O: Storage buckets; `attachments` rows; upload endpoints/helpers.	`T007`	3	P0	Buckets; sto helper; attach
T009	Implement audit logging	E03	S05	DB	BE	Add `audit_log` table and helper (client or API route) to record key actions; verified entries for at least: lead create, cluster assign, quote change, job status change, invoice/payment events.	I/O: `audit_log` inserts; action constants.	`T007`	3	P0	`audit_log` sch ample events
T010	Implement ledger accounting basics	E03	S05	DB	BE	Add `ledger_entries` + minimal account taxonomy; record entries for invoice issued and payment received; list view API/query exists.	I/O: ledger entries referencing invoice/payment.	`T007`	3	P0	Ledger schema basic ledger q
T011	Implement RLS policies Phase 1	E02	S06	DB	BE	Implement RLS for identity + roles + leads/clusters/knocks read/write aligned to Admin/Manager/Rep/Crew; tested with representative users.	I/O: RLS policies for core tables (Phase 1).	`T007`	3	P0	RLS policies; t role users.
T012	Implement RLS policies Phase 2	E02	S06	DB	BE	Extend RLS to quotes/jobs/invoices/payments/ledger/attachments; least privilege for Crew and Rep; block cross-role leakage; tested.	I/O: RLS policies (Phase 2).	`T011`	3	P0	Additional R policy docume

Task ID	Task Name	Epic	Story	Subsystem	Owner	Description / Definition of Done	Inputs/Outputs	Dependencies	Hours	Pri	Deliverables
T013	Implement admin bootstrap + user mgmt support	E02	S07	Backend	BE	Create initial admin bootstrap mechanism (seed/script); ensure roles are assignable; baseline user provisioning works.	I/O: user profile rows; role assignment flow support.	`T007, T011`	2	P0	Bootstrap scr ment support;
T014	Implement web auth UI + protected routes	E02	S07	Web	FE	Login/logout flow using Supabase Auth; protected routes by role; session persistence; unauthorized access blocked (UI + data via RLS).	I/O: web login page; route guards.	`T013`	3	P0	Auth screens; based navigati
T015	Implement mobile auth + secure storage	E02	S07	Mobile	Mobile	Mobile login flow; tokens stored securely; session restore; unauthorized access blocked; minimal error states.	I/O: mobile auth screens; secure token store.	`T013`	2	P0	RN auth UI; se store.
T016	Implement authorization helpers + UI gating	E02	S06	Backend	BE	Shared permission helpers (`can()` style); consistent role checks across web/mobile; unit-tested for key cases.	I/O: shared authz module used by apps.	`T014, T015`	3	P0	Shared authz integration in
T017	Implement web lead map (Mapbox)	E04	S08	Web	FE	Mapbox map renders leads as markers; cluster overlay support (display only); map loads reliably with thousands of points (basic clustering on client or simplified).	I/O: Map page; lead markers; selected lead detail.	`T014, T011`	3	P0	Map screen; le lection UX.
T018	Implement polygon draw/edit + persist	E04	S09	Web	FE	Draw/edit polygons on map; persist polygon GeoJSON to `clusters.boundary_geojson`; reload and edit works; validation for basic shape correctness.	I/O: GeoJSON saved/loaded; polygon editing UI.	`T017, T007`	3	P0	Polygon draw SON validatio
T019	Implement clustering engine API route	E04	S10	Backend	BE	API route clusters leads using business rules (max radius, max items per cluster) and optional polygon boundary; writes clusters and lead memberships/refs; deterministic and testable on sample dataset.	I/O: API endpoint; cluster records; membership updates.	`T018, T007`	4	P0	Clustering end tests for cluster
T020	Implement cluster management UI	E04	S10	Web	FE	Manager/Admin can create/recluster/view clusters; sees counts and assignment status; map displays cluster boundaries and memberships.	I/O: cluster list; recluster action; cluster detail.	`T019, T014`	2	P0	Cluster screen overlays.
T021	Implement lead CRUD UI (manual entry)	E05	S11	Web	FE	Create/edit leads manually with property/address fields + sqft; list/search/filter; reliable pagination; audit log entries created on create/update.	I/O: lead forms; lead list; DB rows in `leads/properties`.	`T014, T009, T011`	3	P0	Lead screens events for lead
T022	Implement cluster assignment to reps	E05	S12	Web	FE	Assign clusters to Sales Reps; rep visibility restricted by RLS; audit logs for assignment changes; assignment reflected in rep view.	I/O: assignment UI; `clusters.assigned_rep_id`.	`T020, T021, T016`	2	P0	Assignment U audit events.
T023	Implement knock outcome logging	E05	S12	Web	FE	Responsive rep screen: list assigned leads, log knock outcomes (enum) with notes; offline-lite caching in browser storage; sync on reconnect; audit log entries on knock create.	I/O: `knocks` inserts; rep UI.	`T021, T022, T009`	3	P0	Rep knock UI offline-lite cach

Task ID	Task Name	Epic	Story	Subsystem	Owner	Description / Definition of Done	Inputs/Outputs	Dependencies	Hours	Pri	Deliverables
T024	Implement quote builder	E05	S13	Web	FE	Quote builder supports price-per-sqft base and editable adjustment line items; totals computed; quote status transitions (draft/sent); audit quote changes.	I/O: `quotes/quote_items` rows; totals; UI.	`T021, T009`	4	P0	Quote UI; calc te e; audit ev
T025	Implement quote acceptance → job creation	E05	S13	Backend	BE	Accept quote to create job; enforce valid state transitions; create job record + initial status; audit transition; idempotent endpoint or transaction-safe logic.	I/O: job created from quote; status history.	`T024, T009`	2	P0	Jo creation ems; audit e
T026	Implement crew management UI	E06	S14	Web	FE	Admin/Manager can CRUD crews and assign crew members (users); RLS ensures crew users see only assigned jobs; audit logs for crew changes.	I/O: `crews/crew_members` rows.	`T014, T016`	2	P0	Crew screens; di events.
T027	Implement scheduling UI + appointment records	E06	S14	Web	FE	Schedule jobs to crews (date/time); create appointment records; update job status to scheduled; audit logs; crew can see schedule.	I/O: `appointments` rows; job status updates.	`T025, T026, T009`	3	P0	Scheduling UI ags status up
T028	Implement RN crew app scaffold + sync cache	E06	S14	Mobile	Mobile	Expo RN app baseline with navigation and Supabase client; cache last synced schedule/jobs locally; read-only works offline for last sync.	I/O: RN app skeleton; local cache store.	`T015, T016`	3	P0	RN app scaffol basic offline re
T029	Implement RN schedule viewer + job detail + Mapbox	E06	S14	Mobile	Mobile	Crew sees assigned schedule and job detail; includes Mapbox map for job location; works with cached data when offline.	I/O: RN screens; Mapbox RN integration.	`T028, T027`	3	P0	Schedule scree cache fallback
T030	Implement RN checklist + job status updates	E06	S14	Mobile	Mobile	Crew checklist completion flow; job status transitions (in_progress/complete); sync to DB with RLS; audit log events on status changes.	I/O: checklist stored (json or table); job status updates.	`T029, T009`	2	P0	Checklist UI; events; basic v
T031	Implement RN photo capture + offline upload queue	E06	S15	Mobile	Mobile	Before/after photo capture; upload to storage; offline queue persists locally; retries with backoff; attachment rows created and visible to web.	I/O: storage objects; `attachments` rows; queue state.	`T029, T008, T009`	5	P0	Camera captur attachment m
T032	Implement web photo gallery + attachments view	E06	S15	Web	FE	Web job detail shows before/after photos and attachments; supports filtering by type; ensures crew uploads appear; basic access control verified.	I/O: gallery UI; attachment queries.	`T031, T014`	2	P0	Jo photo gal permissions va
T033	Integrate Postmark email provider	E07	S16	Integrations	BE	Postmark integration via Next.js API route; templates for core notifications; sandbox send works in dev; errors handled and logged.	I/O: `/api/email/send`; template ids; env secrets.	`T003, T014`	3	P0	Email API rou verification.
T034	Implement invoice lifecycle + notifications	E07	S16	Web	FE	Generate invoice from job/quote snapshot; send invoice email; invoice status transitions; payment link display; audit invoice events; ledger entry for invoice issuance.	I/O: `invoices` rows; email sends; ledger entry insert.	`T025, T033, T010, T009`	5	P0	Invoice UI; i email trigger; audit events.
T035	Integrate Stripe payments + reconciliation	E07	S16	Integrations	BE	Create Stripe Checkout session for invoice; webhook handler records payment and updates invoice; ledger entry for payment received; reconciliation view/query works.	I/O: `/api/stripe/*`; `payments` rows; invoice updates; ledger entries.	`T034, T033, T010`	6	P0	Stripe session dle; payment ing; reconciliat

Task ID	Task Name	Epic	Story	Subsystem	Owner	Description / Definition of Done	Inputs/Outputs	Dependencies	Hours	Pri	Deliverables
T036	Implement Google Calendar OAuth + appointment sync	E08	S17	Integrations	BE	OAuth connect for Manager/Admin; store refresh token securely; one-way sync create/update for appointment events; resilience on token refresh; audit sync actions.	I/O: Google tokens; `appointments.calendar_event_id`; API route.	`T027, T014, T003`	4	P0	OAuth connec age appointm tries.
T037	Implement operations dashboard (Nova)	E08	S18	Web	FE	Minimal Admin/Manager dashboard: counts and quick links (leads by status, jobs by status, invoices aging, recent payments); uses existing queries; loads fast.	I/O: dashboard page; summary queries.	`T021, T027, T034, T035`	3	P0	Dashboard p quick filters.
T038	Implement observability basics	E08	S19	DevOps	DevOps	Structured logging for API routes; consistent error handling; minimal error capture strategy (e.g., log + persistent record for critical integration failures); documented on-call checks.	I/O: logger module; integration error logs.	`T006, T033, T035`	2	P0	Logger utility; tegrations; ops
T039	Implement smoke tests + release runbook	E08	S19	DevOps	DevOps	Documented smoke test checklist across workflows A/B/D/E; basic automated sanity checks where feasible; production checklist; rollback plan; runbook.	I/O: checklist; scripts; docs.	`T034, T035, T036, T031, T038`	3	P0	Smoke test ch rollback plan;
T040	Implement mobile sales rep app (optional)	E09	S20	Mobile	Mobile	RN rep app for assigned clusters/leads and knock logging; stronger offline caching than web responsive; background sync.	I/O: RN rep screens; cached datasets; `knocks`.	`T023, T028`	10	P1	RN rep mod sync strategy;
T041	Add CSV import + bulk geocoding pipeline	E09	S21	Backend	BE	Import leads from CSV; geocode addresses in batch; dedupe rules; audit import results; admin-only.	I/O: import endpoint; import report; updated leads/properties.	`T021, T007`	8	P1	CSV import t tion; import a
T042	Expand dashboards & reporting	E09	S22	Web	FE	Add richer dashboards: rep performance, cluster conversion, revenue trends; export CSV; role-based visibility.	I/O: reporting pages; export endpoints.	`T037`	6	P1	Reporting pag missions.
T043	Add PDF invoices + customer portal	E09	S22	Web	FE	Generate PDFs for invoices; optional lightweight customer portal for invoice viewing and payment; improved branding.	I/O: PDF generation; portal pages; secure links.	`T034, T033`	8	P1	PDF invoices; links.
T044	Add advanced observability (Sentry)	E09	S23	DevOps	DevOps	Integrate Sentry (or equivalent) for web/mobile/API; alerts for Stripe/email/calendar failures; performance traces where useful.	I/O: error events; alerts.	`T038`	4	P1	Sentry config;
T045	Expand automated test suite (E2E)	E09	S23	DevOps	DevOps	Add E2E tests for key workflows; CI gates for regressions; test data seeding improvements.	I/O: E2E tests; CI integration.	`T039`	8	P1	E2E tests; tures/seeds.
T046	Multi-tenant architecture	E09	S24	Backend	BE	Introduce tenant/org model; migrate schema and RLS to tenant-scoped; admin tooling for tenants; ensures isolation.	I/O: tenant tables; RLS updates; migration scripts.	`T007, T011, T012`	10	P2	Tenant schem RLS; admin to
T047	Route/day-plan optimization	E09	S24	Backend	BE	Add route/day plan optimization (explicitly not required for MVP); integrate with clusters and job scheduling.	I/O: route plans; UI views.	`T019, T029`	12	P2	Routing engin uation metrics
T048	SMS/Calling integration	E09	S24	Integrations	BE	Add SMS/calling integration (explicitly out of MVP); configurable provider; opt-in templates and logs.	I/O: messaging endpoints; message logs.	`T033`	8	P2	Messaging ser logs; controls.

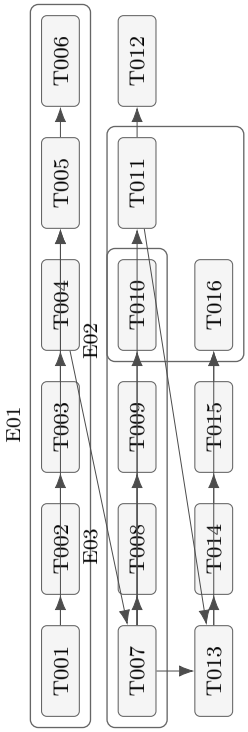
Task ID	Task Name	Epic	Story	Subsystem	Owner	Description / Definition of Done	Inputs/Outputs	Dependencies	Hours	Pri	Deliverables
T049	Offline-first support web	E09	S24	Web	FE	Add offline-first capabilities to web app (explicitly not required for MVP); caching, sync, conflict resolution.	I/O: service worker; sync layer.	`T014, T006`	10	P2	Offline cache; s do.

Dependency Graphs (DAG)

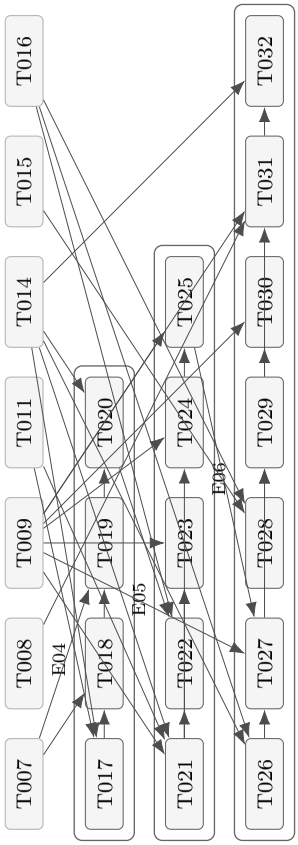
Compact Dependency Cross-Check Table (Complete)

Task	Dependencies (Task ID → deps)
T001	T001 → (none)
T002	T002 → T001
T003	T003 → T001
T004	T004 → T003
T005	T005 → T002, T003
T006	T006 → T005
T007	T007 → T004
T008	T008 → T007
T009	T009 → T007
T010	T010 → T007
T011	T011 → T007
T012	T012 → T011
T013	T013 → T007, T011
T014	T014 → T013
T015	T015 → T013
T016	T016 → T014, T015
T017	T017 → T014, T011
T018	T018 → T017, T007
T019	T019 → T018, T007
T020	T020 → T019, T014
T021	T021 → T014, T009, T011
T022	T022 → T020, T021, T016
T023	T023 → T021, T022, T009
T024	T024 → T021, T009
T025	T025 → T024, T009
T026	T026 → T014, T016
T027	T027 → T025, T026, T009
T028	T028 → T015, T016
T029	T029 → T028, T027
T030	T030 → T029, T009
T031	T031 → T029, T008, T009
T032	T032 → T031, T014
T033	T033 → T003, T014
T034	T034 → T025, T033, T010, T009
T035	T035 → T034, T033, T010
T036	T036 → T027, T014, T003
T037	T037 → T021, T027, T034, T035
T038	T038 → T006, T033, T035
T039	T039 → T034, T035, T036, T031, T038
T040	T040 → T023, T028
T041	T041 → T021, T007
T042	T042 → T037
T043	T043 → T034, T033
T044	T044 → T038
T045	T045 → T039
T046	T046 → T007, T011, T012
T047	T047 → T019, T029
T048	T048 → T033
T049	T049 → T014, T006

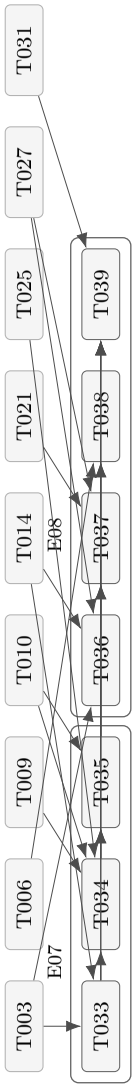
Full MVP DAG (Part 1 of 4): Foundations + Schema + RLS + Auth (T001–T016)


E01
T001
T002
T003
T004
T005
T006
E03
E02
T007
T008
T009
T010
T011
T012
T013
T014
T015
T016

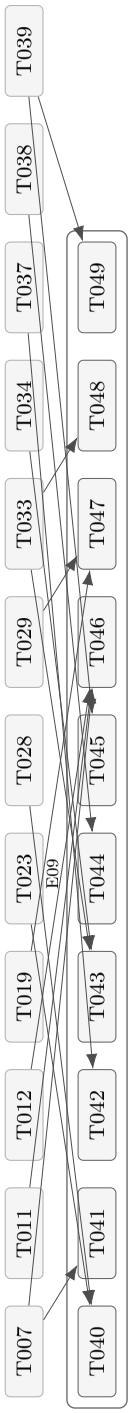
Full MVP DAG (Part 2 of 4): Maps + Sales + Ops Core (T017–T032)


T007
T008
T009
T011
T014
T015
T016
E04
T017
T018
T019
T020
E05
T021
T022
T023
T024
T025
E06
T026
T027
T028
T029
T030
T031
T032

Full MVP DAG (Part 3 of 4): Billing + Calendar + Release (T033–T039)


T003
T006
T009
T010
T014
T021
T025
T027
T031
E07
E08
T033
T034
T035
T036
T037
T038
T039

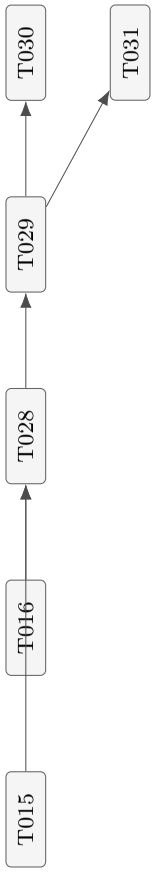
Full MVP DAG (Part 4 of 4): MVP2 Backlog (T040–T049)


T007
T011
T012
T019
T023
T028
T029
T033
T034
T037
T038
T039
E09
T040
T041
T042
T043
T044
T045
T046
T047
T048
T049

Subsystem DAGs (Summary Views)

Note: The full DAG parts above are the source of truth for complete edge coverage. The following diagrams are simplified subsystem views for readability.

Mobile Subsystem DAG (T015, T028–T031)



Milestones & Timeline

Capacity Window and Sprint Framing

- **Start date:** Monday, Feb 16, 2026
- **Target MVP date:** March 1, 2026
- **Capacity assumption:** 2 devs × 6 productive hours/day × 10 weekdays (Feb 16–Feb 27) = **120 hours**
- **Sprint 0 / Setup (Feb 16–Feb 20):** foundations + schema + auth + maps/clustering primitives
- **Sprint 1 / Build + Integrate (Feb 23–Feb 27):** end-to-end workflows + integrations + crew app + billing + release readiness
- **Release / Stabilization buffer (Feb 28–Mar 1):** optional hardening (do not assume capacity; treat as risk response/overtime only)

Milestones with Task IDs (Planned)

Milestone	Date	Target Tasks (exact IDs)
M0: Sprint 0 Complete	Feb 20, 2026	`T001--T020` (repo/tooling, env, Supabase, schema, storage, audit/ledger, RLS, auth, web map, polygon draw, clustering API, cluster UI)
M1: Sprint 1 Feature Complete	Feb 27, 2026	`T021--T039` (leads, assignments, knocks, quotes, jobs, scheduling, crew RN app, photos, email, invoices, Stripe, Calendar sync, dashboard, observability, smoke tests/runbook)
M2: Release (Risk-Managed Buffer)	Mar 1, 2026	Bug fixes, data validation, integration retries; run the smoke checklist; production deploy confirmation.

Critical Path (Gating Chain)

The following ordered chain is the critical path because it unlocks secure data access, core workflow persistence, and integration correctness:

`T001` → `T003` → `T004` → `T007` → `T011` → `T013` → `T014` → `T021` → `T024` → `T025` → `T027` → `T028` → `T031` → `T033`

Why gating: schema+RLS+auth enable safe CRUD; leads/quotes/jobs enable operational workflow; mobile + photos enable crew completion; email + invoices + Stripe + Calendar enable revenue and operations; smoke tests gate release confidence.

MVP Cutline for March 1 (Capacity Check + Contingency)

P0 total hours: 120 hours. **Capacity:** 120 hours. **Result:** Fits exactly, with minimal slack.

Contingency cutline (drop/reduce first if risk materializes):

- **Cut 1:** `T037` (Ops dashboard) → defer to MVP2; replace with existing list views (leads/jobs/invoices).

- **Cut 2:** `T038` (Observability basics) → reduce to "integration error logging only" and defer structured improvements.
- **Cut 3:** Reduce `T036` scope to "create events only" in the short-term; add update/delete sync in MVP2 (retain scheduling in-app). *(Only if necessary; integration risk acknowledged.)*

Effort Summary

Effort by Priority

Priority	Hours	Notes
P0 (March 1 target)	120	Exactly matches stated capacity window
P1 (MVP2 backlog)	44	Enhancements; not required for March 1
P2 (Later)	40	Multi-tenant and advanced optimizations
Total (All Tasks)	**204**	Full roadmap effort

P0 Effort by Epic

Epic	Hours (P0)	Comments
E01	18	Repo, env, CI, deploy
E03	14	Schema, storage, audit, ledger
E02	16	RLS, auth, permission helpers
E04	12	Mapbox, polygons, clustering
E05	14	Sales workflow A/B + quote-to-job
E06	20	Scheduling + crew mobile + photos
E07	14	Email, invoices, Stripe payments
E08	12	Calendar sync, dashboard, observability, release
Total	**120**	

P0 Effort by Subsystem

Subsystem	Hours (P0)	Coverage
DevOps	23	Tooling, CI/CD, deploy, observability, release
DB	17	Schema, RLS, audit, ledger
Backend	11	Authz helpers, clustering API, job creation
Web	38	Lead/cluster/quote/schedule/invoice/dashboard UIs
Mobile	15	Crew app, checklist, photos, offline upload queue
Integrations	16	Storage, Postmark, Stripe, Google Calendar
Total	**120**	

Machine-readable Dependency Block

Valid JSON (source of truth for dependencies). Fields: `id, name, epic, story, subsystem, owner, hours, priority, deps`.


```
{
  "version": "1.0",
  "generated_on": "2026-02-08",
  "tasks": [
    {"id":"T001","name":"Initialize monorepo","epic":"E01","story":"S01","subsystem":"DevOps",'
    {"id":"T002","name":"Configure code quality toolchain","epic":"E01","story":"S01","subsyste
    {"id":"T003","name":"Set up env/secrets patterns","epic":"E01","story":"S02","subsystem":"I
    {"id":"T004","name":"Provision Supabase + local dev","epic":"E01","story":"S02","subsystem'
    {"id":"T005","name":"Set up CI + preview deploy","epic":"E01","story":"S02","subsystem":"De
    {"id":"T006","name":"Configure production deploy","epic":"E01","story":"S02","subsystem":"I

    {"id":"T007","name":"Implement core DB schema migrations","epic":"E03","story":"S03","subsy
    {"id":"T008","name":"Implement storage + attachments upload flow","epic":"E03","story":"S04
    {"id":"T009","name":"Implement audit logging","epic":"E03","story":"S05","subsystem":"DB","
    {"id":"T010","name":"Implement ledger accounting basics","epic":"E03","story":"S05","subsys

    {"id":"T011","name":"Implement RLS policies Phase 1","epic":"E02","story":"S06","subsystem'
    {"id":"T012","name":"Implement RLS policies Phase 2","epic":"E02","story":"S06","subsystem'

    {"id":"T013","name":"Implement admin bootstrap + user mgmt support","epic":"E02","story":"S
    {"id":"T014","name":"Implement web auth UI + protected routes","epic":"E02","story":"S07","
    {"id":"T015","name":"Implement mobile auth + secure storage","epic":"E02","story":"S07","su
    {"id":"T016","name":"Implement authorization helpers + role-based UI gating","epic":"E02","

    {"id":"T017","name":"Implement web lead map (Mapbox)","epic":"E04","story":"S08","subsystem
    {"id":"T018","name":"Implement polygon draw/edit + persist","epic":"E04","story":"S09","sub
    {"id":"T019","name":"Implement clustering engine API route","epic":"E04","story":"S10","sub
    {"id":"T020","name":"Implement cluster management UI","epic":"E04","story":"S10","subsystem

    {"id":"T021","name":"Implement lead CRUD UI (manual entry)","epic":"E05","story":"S11","sub
    {"id":"T022","name":"Implement cluster assignment to reps","epic":"E05","story":"S12","subs
    {"id":"T023","name":"Implement knock outcome logging","epic":"E05","story":"S12","subsystem
    {"id":"T024","name":"Implement quote builder","epic":"E05","story":"S13","subsystem":"Web"
    {"id":"T025","name":"Implement quote acceptance -> job creation","epic":"E05","story":"S13'

    {"id":"T026","name":"Implement crew management UI","epic":"E06","story":"S14","subsystem":'
    {"id":"T027","name":"Implement scheduling UI + appointment records","epic":"E06","story":"S
    {"id":"T028","name":"Implement RN crew app scaffold + sync cache","epic":"E06","story":"S14
    {"id":"T029","name":"Implement RN schedule viewer + job detail + Mapbox","epic":"E06","sto
    {"id":"T030","name":"Implement RN checklist + job status updates","epic":"E06","story":"S14
    {"id":"T031","name":"Implement RN photo capture + offline upload queue","epic":"E06","story
    {"id":"T032","name":"Implement web photo gallery + attachments view","epic":"E06","story":'
```

```
    {"id":"T033","name":"Integrate Postmark email provider","epic":"E07","story":"S16","subsys
    {"id":"T034","name":"Implement invoice lifecycle + notifications","epic":"E07","story":"S16
    {"id":"T035","name":"Integrate Stripe payments + reconciliation","epic":"E07","story":"S16

    {"id":"T036","name":"Implement Google Calendar OAuth + appointment sync","epic":"E08","sto
    {"id":"T037","name":"Implement operations dashboard (Nova)","epic":"E08","story":"S18","su
    {"id":"T038","name":"Implement observability basics","epic":"E08","story":"S19","subsystem
    {"id":"T039","name":"Implement smoke tests + release runbook","epic":"E08","story":"S19","

    {"id":"T040","name":"Implement mobile sales rep app (optional)","epic":"E09","story":"S20"
    {"id":"T041","name":"Add CSV import + bulk geocoding pipeline","epic":"E09","story":"S21",
    {"id":"T042","name":"Expand dashboards & reporting","epic":"E09","story":"S22","subsystem"
    {"id":"T043","name":"Add PDF invoices + customer portal","epic":"E09","story":"S22","subsys
    {"id":"T044","name":"Add advanced observability (Sentry)","epic":"E09","story":"S23","subs
    {"id":"T045","name":"Expand automated test suite (E2E)","epic":"E09","story":"S23","subsys
    {"id":"T046","name":"Multi-tenant architecture","epic":"E09","story":"S24","subsystem":"Ba
    {"id":"T047","name":"Route/day-plan optimization","epic":"E09","story":"S24","subsystem":"
    {"id":"T048","name":"SMS/Calling integration","epic":"E09","story":"S24","subsystem":"Inte
    {"id":"T049","name":"Offline-first web support","epic":"E09","story":"S24","subsystem":"We
  ]
}
```


Appendix

Estimation Assumptions

- Hour estimates include: implementation, basic unit tests (where noted), code review fixes, and minimal documentation updates.
- Estimates assume heavy reuse of managed services (Supabase Auth/RLS/Storage, Stripe Checkout, Postmark templates) and minimal UI polish for MVP.
- The plan is **capacity-tight** (P0 = 120 hours exactly). Any integration surprises (Stripe webhook edge cases, Google OAuth friction, mobile build issues) can create schedule risk.

Key Risks and Mitigations

- **Google OAuth/Calendar complexity:** token storage and refresh issues. Mitigation: implement one-way sync first; add robust logging; keep manual scheduling as source of truth.
- **Mobile photo uploads and offline queue:** device permissions, network variability. Mitigation: keep queue simple and persistent; test on 2 devices; keep upload sizes bounded.
- **RLS correctness:** subtle data leakage risks. Mitigation: create a role-based verification matrix and run before release; keep schema and policies small.
- **Exact-capacity plan:** minimal slack. Mitigation: cutline (defer dashboard/observability polish), focus on end-to-end workflow integrity.

Operational Readiness Checklist (Minimum)

- Confirm prod env vars for Supabase/Mapbox/Stripe/Postmark/Google
- Verify Stripe webhook endpoint and signing secret
- Verify Postmark sender domain and template IDs
- Verify Google OAuth consent screen + redirect URIs
- Run smoke tests for workflows A/B/D/E
- Confirm RLS policy matrix for each role